

BCSE408L – Cloud Computing

Initial Software Requirements Specification (SRS)

LocalShop Cloud

Digital Storefront for Local Retailers

Project Category: Retail SaaS

Review 1 Submission

Team Members

Abinayaa M B – 23BCE1913

Maneesh Ram K – 23BCE5018

School of Computer Science Engineering and Information Systems

VIT Chennai | Academic Year 2026–2027

1. Introduction

LocalShop Cloud is a cloud-native Software-as-a-Service (SaaS) application built for the Retail SaaS domain. It provides local retailers — such as neighbourhood grocery stores, boutiques and small businesses — with a lightweight, affordable digital storefront through which they can list products, manage inventory and receive orders from nearby customers, without depending on large commission-heavy marketplaces. The system is designed with three primary roles: Retailer, Customer and Admin, and will be deployed as a fully cloud-hosted, publicly accessible application using a cloud database, REST APIs and a responsive front end.

This document defines the requirements of the LocalShop Cloud system. It describes the problem being addressed, existing alternatives, the proposed solution, project scope, and the functional and non-functional requirements that guide the system's design, development, and implementation.

2. Problem Statement

Local retailers largely lack an affordable, easy way to sell online. Most are limited to one of three options, each with significant drawbacks:

- Listing on large marketplaces (e.g., Amazon, Flipkart), which charge high commissions, bury small sellers in search rankings, and offer little brand identity.
- Selling informally through WhatsApp or social media, which has no structured cart, inventory tracking, order management or payment record.
- Having no online presence at all, restricting sales to walk-in customers within physical reach of the store.

There is a need for a low-friction, cloud-hosted storefront platform that lets any local retailer set up an online shop quickly, manage their catalog and orders centrally, and reach nearby customers — while giving customers a simple way to discover and order from local shops.

3. Existing System

The current landscape of alternatives available to local retailers includes:

- Large e-commerce marketplaces — high seller commissions (often 10–30%), complex onboarding, and limited control over branding or customer relationships.
- Generic e-commerce website builders (e.g., Shopify-style tools) — capable but often costly for small retailers, with features aimed at larger sellers and steep learning curves.
- Informal channels (WhatsApp Business, Instagram, phone orders) — no centralized inventory or order dashboard, no role-based access, no analytics, and high manual effort to track sales.
- No dedicated, low-cost, cloud-native platform exists that is purpose-built for very small, local retailers with a simple multi-tenant storefront model.

These gaps mean many local retailers remain digitally excluded or absorb high costs to gain a limited online presence.

4. Proposed System

LocalShop Cloud proposes a cloud-native, multi-tenant SaaS platform where each retailer can register and operate their own online storefront under a shared platform, with an admin layer overseeing retailer onboarding and platform health. Key characteristics of the proposed system:

- Retailers can register, manage a product catalog and view/manage incoming orders through a dedicated dashboard.
- Customers can browse storefronts and products, search and filter, add items to a cart, and place orders.
- Admins can approve/manage retailer accounts, monitor platform-wide activity and generate reports.
- The system is deployed on a public cloud platform with a cloud-hosted database, secured REST APIs, and a responsive UI accessible on both desktop and mobile.
- Role-based access control ensures each user type sees only the functionality relevant to their role.

By centralizing storefront creation, inventory, and order handling in one cloud application, LocalShop Cloud aims to give local retailers marketplace-like capability without marketplace-level commissions.

5. Objectives

- Develop a scalable, cloud-native SaaS application for local retail commerce.
- Implement secure, role-based authentication and authorization for Retailer, Customer and Admin roles.
- Provide retailers with dashboards and analytical reports on sales and inventory.
- Deploy the application on a public cloud platform with a live, publicly accessible URL.
- Demonstrate collaborative, industry-standard software engineering practices (version control, documentation, iterative review).

6. Scope

6.1 In Scope

- A multi-tenant storefront model supporting multiple independent retailers on one platform.
- Core retail workflow: product catalog management, browsing, cart, order placement and order tracking.
- Role-based access for Retailer, Customer and Admin.
- Cloud-hosted database and RESTful APIs connecting a responsive front end to the backend.
- Basic reporting/dashboard views (e.g., sales summary, order counts) for retailers and admin.
- Live deployment on a public cloud platform (e.g., Render, Railway, Vercel, AWS, Azure or GCP).

6.2 Out of Scope

- Integrated online payment gateway — payments will be handled as Cash on Delivery / manual confirmation for this phase.

- Third-party logistics or delivery-partner integration.
- Native mobile applications — the platform will be web-based and responsive rather than a dedicated iOS/Android app.
- Multi-language/localization support.

7. Target Users

Role	Description
Retailer	Local shop owner who registers a storefront, manages products/inventory and fulfils orders.
Customer	End user who browses nearby storefronts, searches products and places orders.
Admin	Platform administrator who approves retailers, moderates listings and monitors overall platform activity.

8. Functional Requirements

8.1 Authentication & User Management

- The system shall allow Retailers and Customers to register and log in securely.
- The system shall support role-based access control distinguishing Retailer, Customer and Admin.
- Users shall be able to view and update their profile information.

8.2 Retailer Module

- Retailers shall be able to create, read, update and delete (CRUD) product listings, including name, price, description, category, image and stock quantity.
- Retailers shall be able to view and manage incoming orders (accept, mark fulfilled, cancel).
- Retailers shall have access to a dashboard summarizing sales, order counts and stock levels.

8.3 Customer Module

- Customers shall be able to browse storefronts and search/filter products by category, price or retailer.
- Customers shall be able to add products to a cart and place an order.
- Customers shall be able to view their order history and order status.

8.4 Admin Module

- Admin shall be able to approve, suspend or remove retailer accounts.
- Admin shall be able to view platform-wide analytics (e.g., total retailers, total orders, active users).
- Admin shall be able to moderate product listings if flagged as inappropriate.

8.5 Common Requirements

- The system shall send notifications for key events (e.g., new order placed, order status change).
- The system shall generate reports/exports for sales and order data.

- The system shall integrate with a cloud-hosted database for persistent storage.
- The system shall expose core functionality through REST APIs.

9. Non-Functional Requirements

Attribute	Requirement
Security	User credentials must be securely stored (hashed passwords) and access to data restricted by role-based authorization.
Scalability	The system should support multiple concurrent retailer storefronts and customer sessions without significant performance degradation.
Availability	The application should be reliably accessible via a live public URL with minimal downtime.
Performance	Core pages (catalog browsing, cart, dashboard) should load within an acceptable response time under normal load.
Usability	The UI should be intuitive for non-technical retailers and customers, with a responsive layout for mobile and desktop.
Reliability	Order and inventory data must remain consistent and be reliably recorded in the cloud database.

10. Suggested Modules

- Authentication & User Management
- Retailer Storefront Management (Catalog, Inventory)
- Order & Cart Management
- Reports & Analytics
- Administration

11. Technology Stack (Proposed)

Layer	Proposed Technology
Frontend	React
Backend	Node.js
Database	MongoDB Atlas
Hosting	AWS
Version Control	Git & GitHub (single team repository)

12. Cloud Requirements

- A cloud-hosted database for persistent storage of users, products, orders and reports.
- REST APIs connecting the frontend and backend, secured with authentication tokens.
- Cloud storage for product images where applicable.
- Deployment on a public cloud platform, with the final application accessible only via a live public URL (no localhost-only deployment).

13. Use Case Diagram

The diagram below shows the three actors — Retailer, Customer and Admin — and their respective interactions with the LocalShop Cloud system.

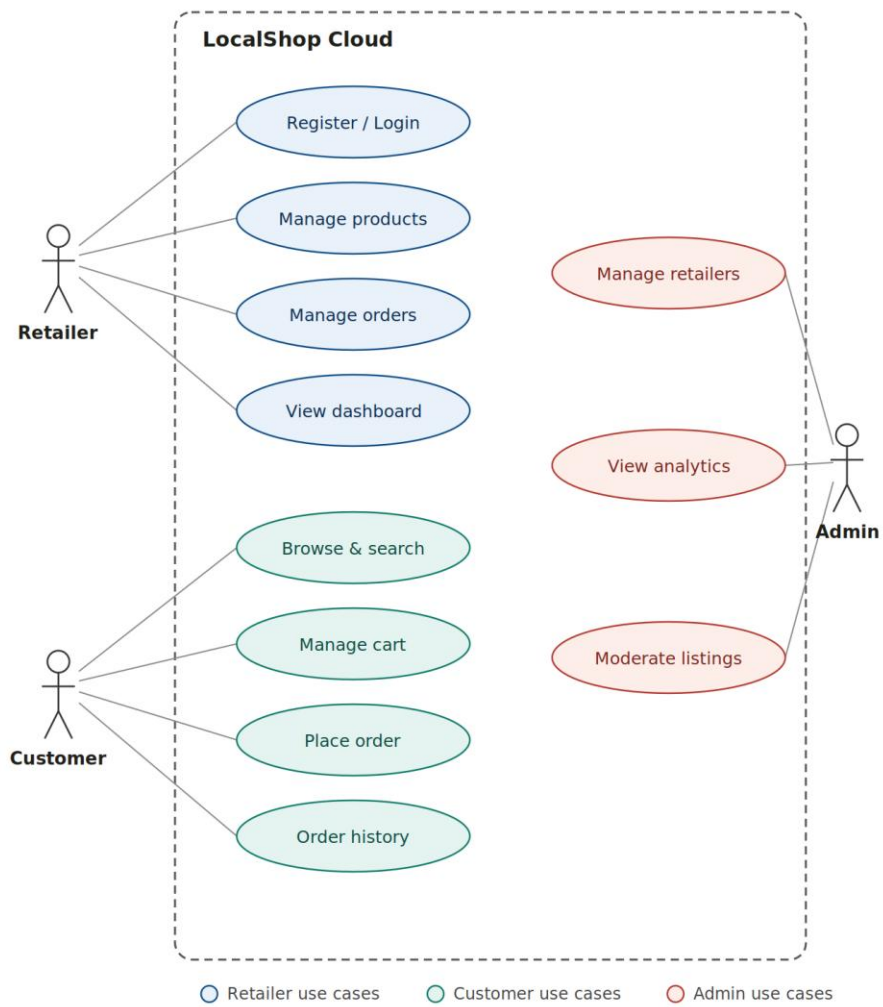


Figure 1: Use Case Diagram — Retailer, Customer and Admin interactions with LocalShop Cloud

14. UI Mockups

Initial UI mockups for the key screens of LocalShop Cloud are shown below, covering the primary workflows for each role.

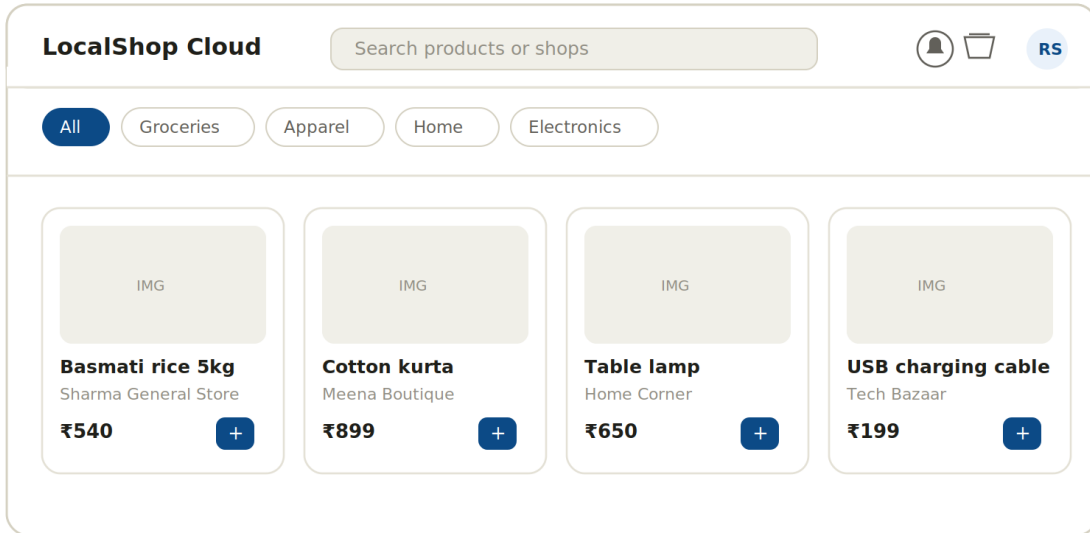


Figure 2: Customer Storefront — search, category filters, product grid

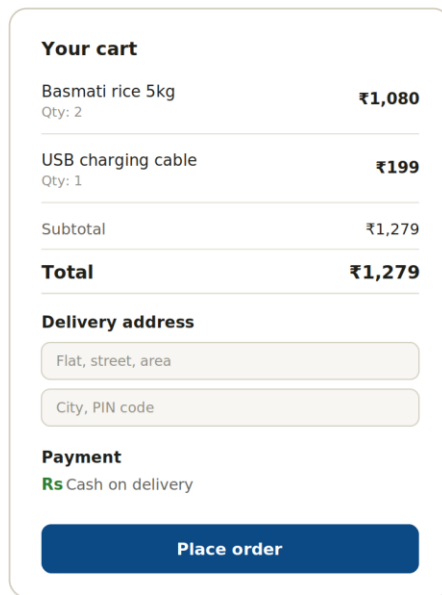


Figure 3: Cart & Checkout — item list, delivery address, order placement

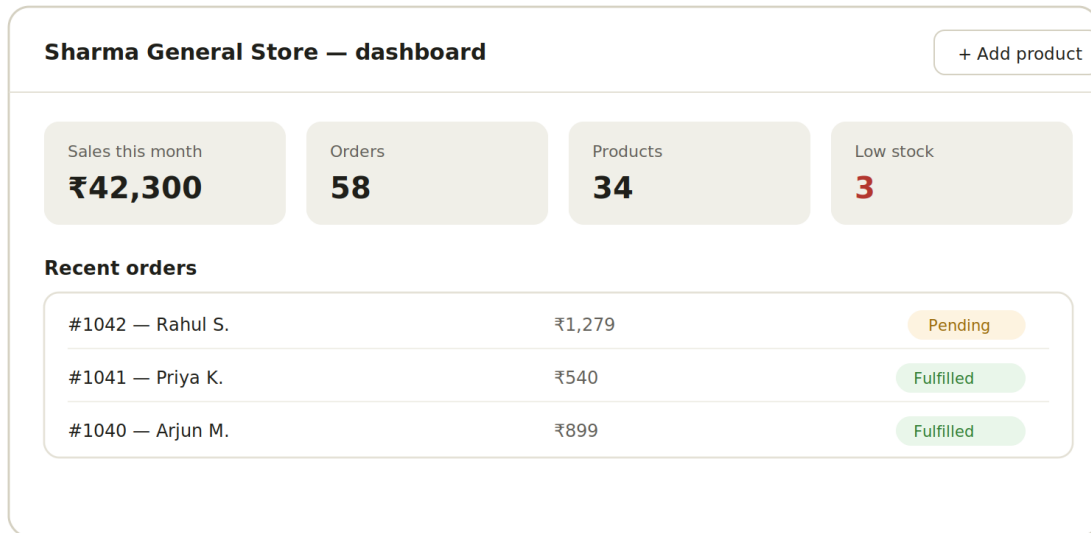


Figure 4: Retailer Dashboard — sales stats, product and order management

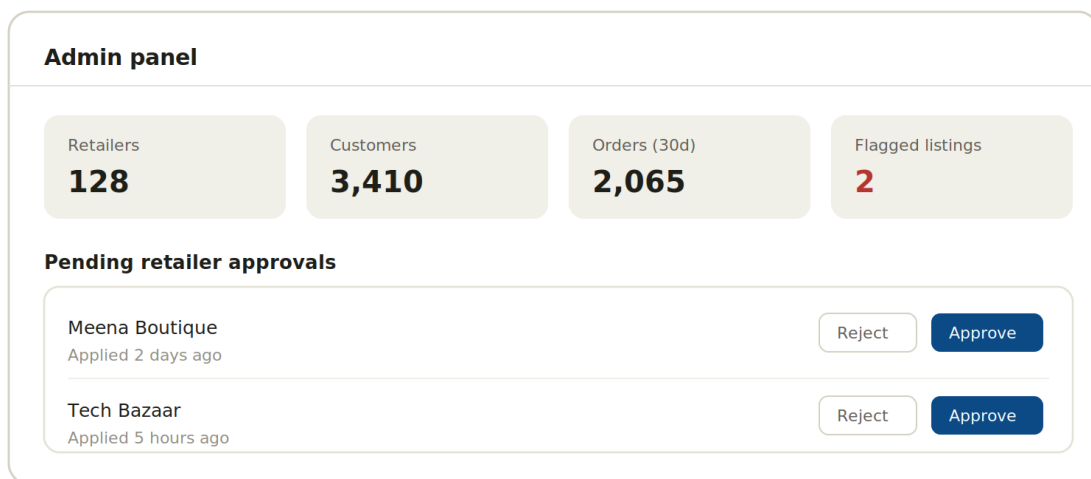


Figure 5: Admin Panel — platform analytics, retailer approval workflow

15. Assumptions & Constraints

- Payment processing is simulated / manual for this phase and not integrated with a live payment gateway.
- The application will be deployed on a free/low-cost tier of the chosen cloud platform, which may impose resource limitations.